IBDA2041 – Proyek – Kelompok IBDA02

Anggota:

- Billy Hartono	232202959	IBDA 2023
- Brian Christian Suroso	232202983	IBDA 2023
- Joey Victor Natanael Djaja	232203245	IBDA 2023

---

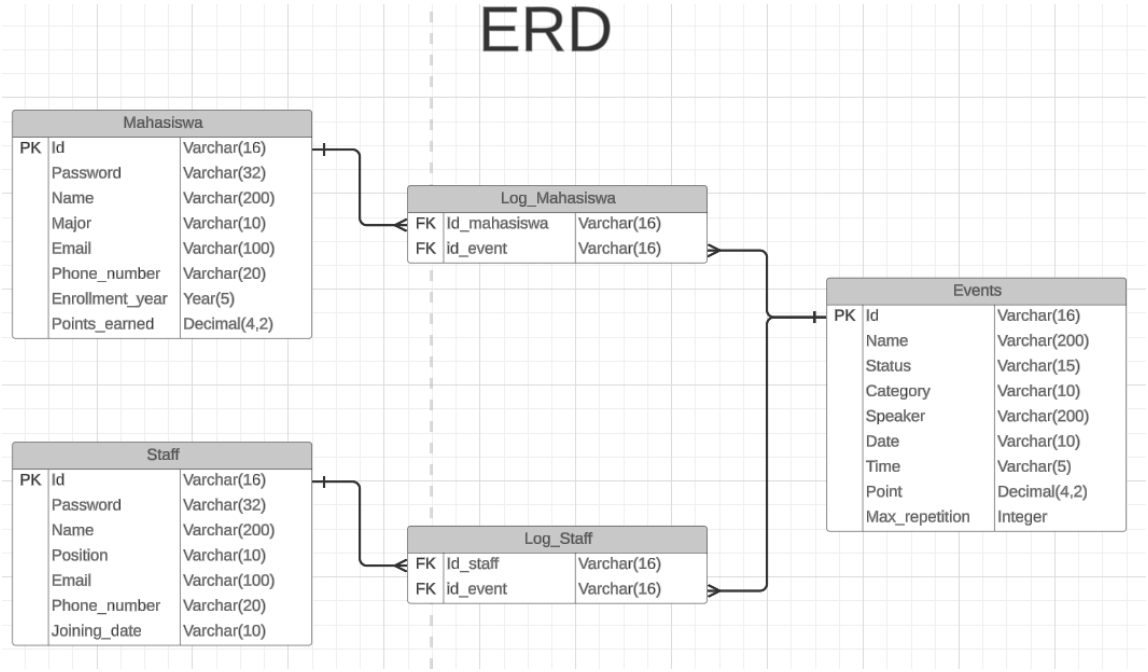
1. Deskripsi Program

Sebagai mahasiswa dari program studi *IT & Big Data Analytics*, kami melihat bahwa pengembangan Sistem Poin Kemahasiswaan (SPK) adalah salah satu proyek yang menarik. Dengan pengembangan proyek ini, kami mengharapkan adanya kemudahan dalam proses input pengelolaan dan rekap data poin mahasiswa. Dari proyek ini kami berharap agar kami dapat lebih memahami bagaimana cara untuk mengimplementasikan pelajaran yang kami dapatkan dengan apa yang dibutuhkan di kehidupan nyata.

Program ini dirancang untuk membuat miniatur/*prototype* dari Sistem Poin Kemahasiswaan. Program ini berbasis *python* dan menggunakan *tkinter* untuk memberikan tampilan kepada pengguna. Program ini juga akan menggunakan berbagai kegunaan dari *mysql* yang akan dihubungkan dengan *python* menggunakan *library pymysql.*

2. Struktur Database

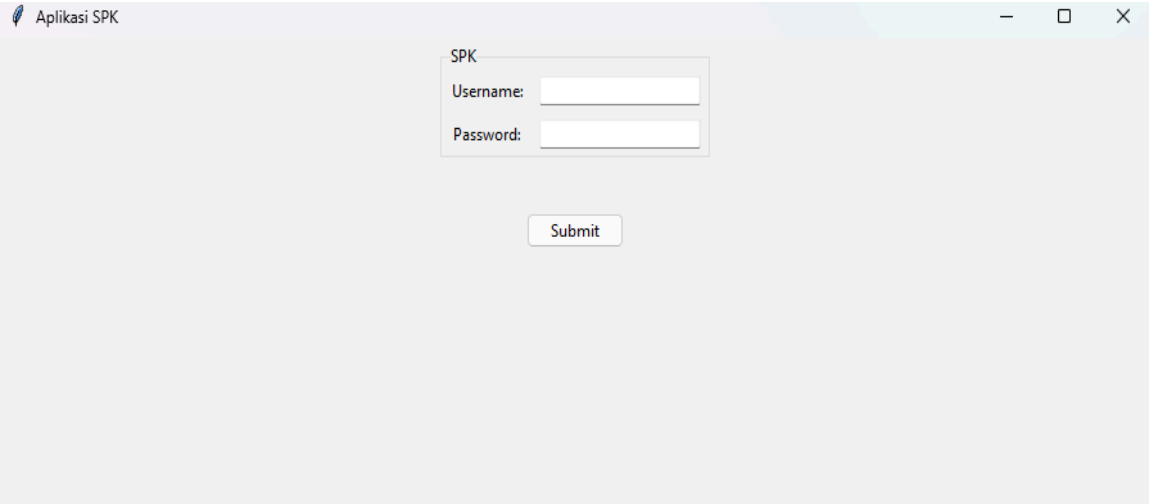
ERD *(Entity-Relationship Diagram):*



Dapat dilihat dari diagram tersebut, kami akan menggunakan 5 buah tabel. Kami menggunakan 3 buah tabel utama yaitu tabel data mahasiswa, *staff*, dan *event*. Kami menambahkan 2 buah tabel tambahan (log mahasiswa dan *staff*) untuk menjadi tabel perantara dari hubungan *many to many* antara tabel mahasiswa dan *events* ataupun tabel *staff* dengan *events*. Diagram tersebut telah disajikan secara *physical* karena hubungan *many to many* telah diberikan tabel perantara (tabel *log*).

3. Tampilan

3.1. Halaman *login*

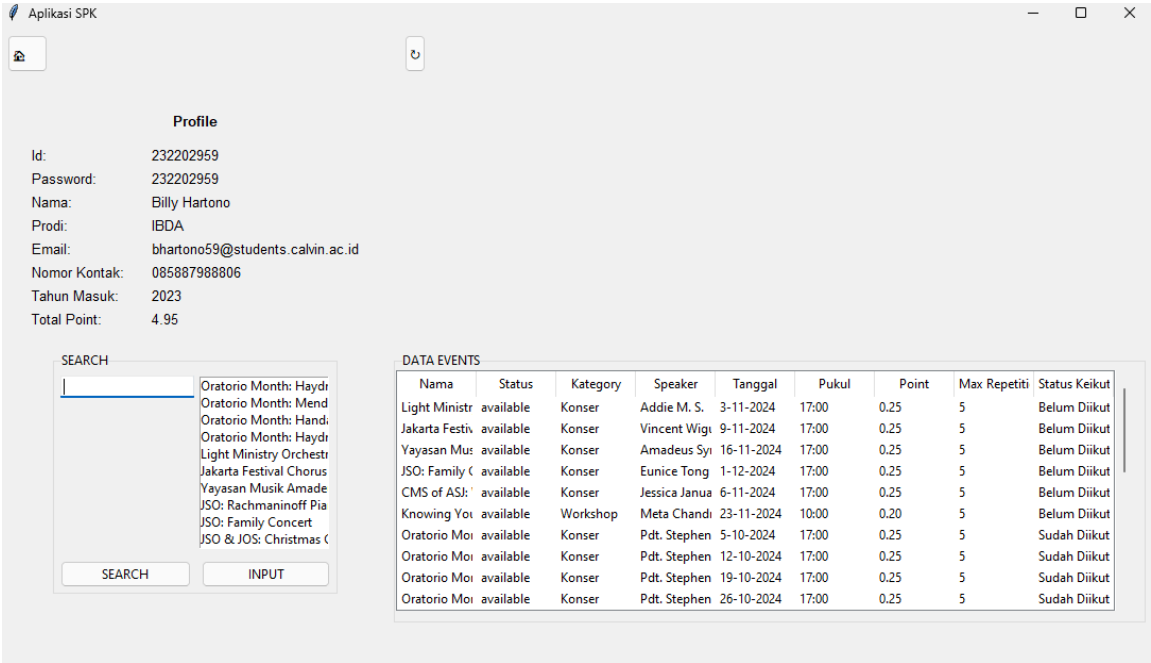


Gambar halaman *login.*

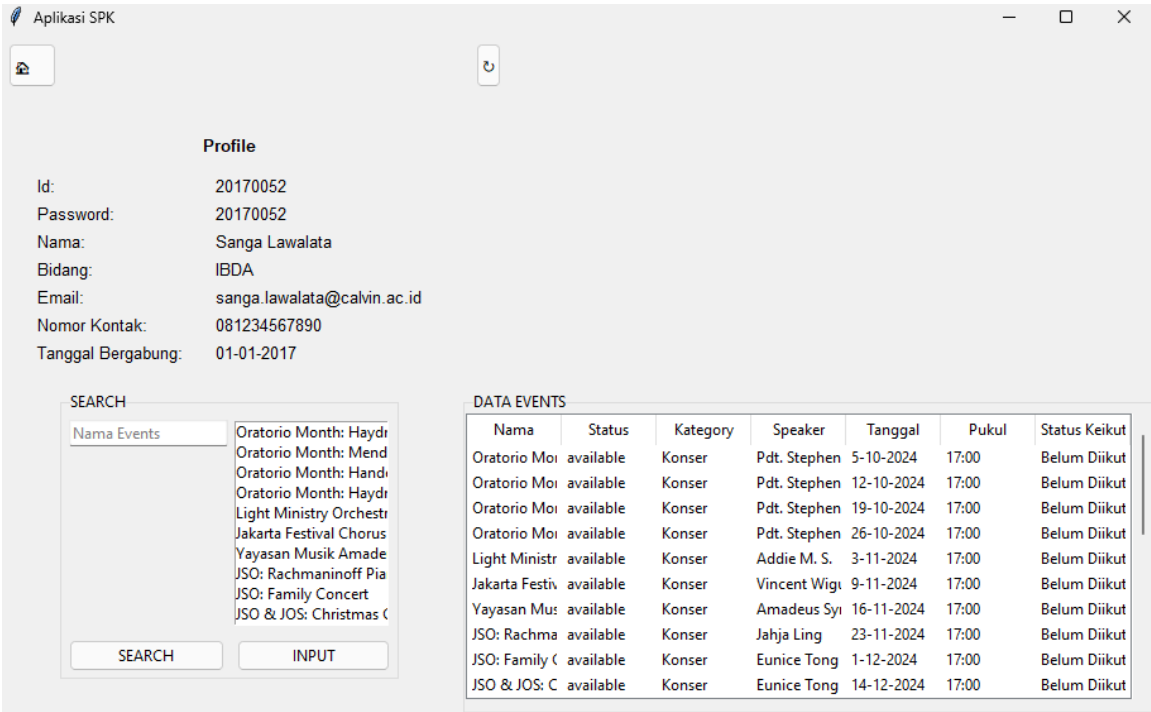
Halaman *login* ini yang akan membedakan pengguna dan memastikan bahwa pengguna asli yang mencoba untuk *login*. Kami menyediakan tiga opsi pengguna, yaitu mahasiswa, *staff,* dan admin. Agar bisa membedakan pengguna yang mencoba *login*, kami mendesain untuk id dari mahasiswa, *staff*, dan admin pasti unik dan tidak mungkin sama. Ketika tombol submit ditekan, pengecekan pertama dilakukan pada *dictionary admin* (*hard coded*) dan kemudian jika tidak id tidak ditemukan baru akan mengakses *database* mahasiswa dan *staff*.

```
def login_validation(username, password):
    if username in adminD and password == adminD[username]:
        frame_login_label_result.config(text="Login berhasil sebagai admin", foreground="green", justify="center")
        frame_login_label_result.config(text="")
        show_frame('admin')
        elif username in (student := {s[0]:s[1] for s in select_items("id, password", "mahasiswa")[0]}) and password ==
student[username]:
        frame_login_label_result.config(text="Login berhasil sebagai mahasiswa", foreground="green", justify="center")
        frame_login_label_result.config(text="")
        show_frame('student')
        load_all_data_events_students()
    elif username in (staff := {s[0]:s[1] for s in select_items("id, password", "staff")[0]}) and password == staff[username]:
        frame_login_label_result.config(text="Login berhasil sebagai staff", foreground="green", justify="center")
        frame_login_label_result.config(text="")
        show_frame('staff')
```

3.2. Halaman mahasiswa/*staff*



Gambar halaman *login* mahasiswa.



Gambar halaman *login staff*

Tampilan dari mahasiswa maupun *staff* menggunakan basis yang cukup mirip. Perbedaannya terletak pada data yang ditampilkan. Dalam tabel mahasiswa terdapat bobot poin suatu acara dan persyaratannya, sedangkan tabel *staff* tidak menampilkan poin namun yang ditampilkan adalah status kehadiran dari sebuah acara. Pada bagian atas setiap halaman terdapat dua tombol yaitu tombol *logout* yang berfungsi untuk keluar dari akun dan *refresh* yang berfungsi sebagai tombol untuk memperbarui data yang ada di dalam tabel, terutama ketika sudah melakukan pencarian. Di bawah tombol *logout* dan *refresh* terdapat profil yang berisikan data-data dari pengguna. Kemudian di bagian paling bawah terdapat informasi kegiatan baik yang sudah diikuti ataupun belum diikuti. Berikut adalah kode dari halaman tersebut per bagian beserta dengan fungsinya:

- Inisialisasi penempatan elemen dari elemen (seperti tombol, *entry, listbox,* tabel, dan sebagainya)

```
# Tombol dan input
frame_student_home_button = ttk.Button(frame_student, text='Log Out', width=0, command=lambda: show_frame('login'))
frame_student_home_button.grid(row=0, column=0, padx=10, ipady=5, pady=5, sticky='w')

frame_show_students_home_button = ttk.Button(frame_student, text='↻', width=0, command=load_all_data_events_students)
frame_show_students_home_button.grid(row=0, column=1, padx=10, ipady=5, pady=5, sticky='w')

entry_var_events_students = tk.StringVar()
options_events_students = get_data_listbox_from_mysql_events_students('')
frame_student_input = ttk.LabelFrame(frame_student, text='SEARCH')
frame_student_input.grid(row=2, column=0, sticky='n')

frame_student_name_entry = ttk.Entry(frame_student_input, textvariable=entry_var_events_students, width=20)
frame_student_name_entry.grid(row=0, column=0, padx=(5, 2), pady=5, sticky="n")
frame_student_name_entry.insert(0, "Nama Events")
frame_student_name_entry.config(foreground='grey')
frame_student_name_entry.bind("<FocusIn>", lambda event: on_click(event, frame_student_name_entry))
frame_student_name_entry.bind("<FocusOut>", lambda event: on_focusout(event, frame_student_name_entry, 'Nama Events'))

frame_student_listbox = tk.Listbox(frame_student_input, width=20, selectmode=tk.SINGLE)
frame_student_listbox.grid(row=0, column=1, padx=(2, 5), pady=5, sticky="ew")

frame_student_name_entry.bind("<KeyRelease>",                                lambda                                event:
filter_options_events_students(entry_var_events_students.get(), frame_student_listbox))
frame_student_listbox.bind("<<ListboxSelect>>", lambda event: on_select(event, entry_var_events_students))

frame_student_submit_button = ttk.Button(frame_student_input, text='SEARCH', command=filtering_events_students)
frame_student_submit_button.grid(row=1, column=0, padx=5, pady=5, sticky='ew')

frame_student_submit_button = ttk.Button(frame_student_input, text='INPUT', command=input_to_log_students)
frame_student_submit_button.grid(row=1, column=1, padx=5, pady=5, sticky='ew')

frame_student_Label = ttk.LabelFrame(frame_student, text='DATA EVENTS')
frame_student_Label.grid(row=2, column=1, ipadx=5, ipady=5)

columns  =  ('Nama',  'Status',  'Kategory',  'Speaker',  'Tanggal',  'Pukul',  'Point',  'Max  Repetition',  'Status
Keikutsertaan')
```

```python
frame_student_tree = ttk.Treeview(frame_student_Label, columns=columns, show="headings")
frame_student_tree.grid(row=0, column=0)

for col in columns:
    frame_student_tree.heading(col, text=col)
    frame_student_tree.column(col, width=75)

tree_scroll = ttk.Scrollbar(frame_student_Label, orient="vertical", command=frame_student_tree.yview)
tree_scroll.grid(row=0, column=1, sticky='ns')
frame_student_tree.config(yscrollcommand=tree_scroll.set)

load_all_data_events_students()
```

- Bagian untuk menghasilkan kemampuan filter dan search menggunakan *listbox*:

```python
def filter_options_events_students(event, jenis):
    typed_text = entry_var_events_students.get()
    filtered_options = [option for option in options_events_students if typed_text.lower() in option[0].lower()]
    frame_student_listbox.delete(0, tk.END)
    for option in filtered_options:
        frame_student_listbox.insert(tk.END, option[0])

def filtering_events_students():
    name_filter = entry_var_events_students.get().lower()
    data = get_data_from_mysql_events_students(name_filter)
    display_data_events_students(data)
```

Untuk melakukan proses filter, setiap karakter yang masuk akan disimpan dan dicari ke dalam *database* dengan menggunakan filter tersebut. Kemudian semua tampilan yang ada akan dihapuskan dan kemudian ditampilkan data yang sudah difilter tersebut.

- Bagian untuk mengambil data dari *database*:

```python
def get_data_from_mysql_events_students(name_filter):
    student_id = frame_login_entry_username.get()
    try:
        query = '''
                SELECT
                    e.name, e.status, e.category, e.speaker, e.date, e.time, e.point, e.max_repetition,
                    CASE
                        WHEN se.id_event IS NOT NULL THEN 'Sudah Diikuti'
                        ELSE 'Belum Diikuti'
                    END AS status_tamb
                FROM events AS e
                LEFT JOIN log_mahasiswa AS se
                ON e.id = se.id_event AND se.id_mahasiswa = %s
                WHERE e.name LIKE %s
                ORDER BY
                    status_tamb ASC,
                    e.status ASC,
                    e.id ASC;
                '''
        cursor.execute(query, (student_id, '%' + name_filter + '%'))
        rows = cursor.fetchall()
        return rows if rows else []
    except Exception as e:
        print(f"Error fetching filtered data: {e}")
        return []

def get_data_listbox_from_mysql_events_students(name_filter):
    student_id = frame_login_entry_username.get()
```

```python
    try:
        query = '''
                SELECT
                    e.name, e.status, e.category, e.speaker, e.date, e.time, e.point, e.max_repetition,
                    CASE
                        WHEN se.id_event IS NOT NULL THEN 'Sudah Diikuti'
                        ELSE 'Belum Diikuti'
                    END AS status_tamb
                FROM events AS e
                LEFT JOIN log_mahasiswa AS se
                ON e.id = se.id_event AND se.id_mahasiswa = %s
                WHERE e.name LIKE %s AND e.status = 'available'
                ORDER BY
                    status_tamb ASC,
                    e.status ASC,
                    e.id ASC;
                '''
        cursor.execute(query, (student_id, '%' + name_filter + '%'))
        rows = cursor.fetchall()
        listbox_data = [row for row in rows if row[-1] == "Belum Diikuti"]
        return rows if rows else []
    except Exception as e:
        print(f"Error fetching filtered data: {e}")
        return []
```

- Bagian untuk memasukkan ke dalam tabel *log*:

```python
def input_to_log_students():
    student_id = frame_login_entry_username.get()
    event_name = entry_var_events_students.get().strip().lower()  # Ambil nama event dari input
    query_event_details = 'SELECT id, status, point FROM events WHERE LOWER(name) = %s'
    cursor.execute(query_event_details, (event_name,))
    event_details = cursor.fetchone()
    print(event_details)

    if event_details:  # Jika event ditemukan di database
        event_id, event_status, event_point = event_details

        # Cek apakah event sudah diikuti oleh mahasiswa
        query_check_log = 'SELECT 1 FROM log_mahasiswa WHERE id_mahasiswa = %s AND id_event = %s'
        cursor.execute(query_check_log, (student_id, event_id))
        log_exists = cursor.fetchone()

        if log_exists:  # Jika sudah ada di log
            messagebox.showwarning("Peringatan", "Event ini sudah pernah diikuti oleh Anda.")
        elif event_status.lower() != "available":  # Jika status event tidak tersedia
            messagebox.showwarning("Peringatan", "Event tidak tersedia untuk diikuti.")
        else:  # Jika event tersedia dan belum diikuti, tambahkan ke log
            try:
                # Tambahkan ke log mahasiswa
                insert_values("log_mahasiswa", "id_mahasiswa, id_event", [(student_id, event_id)])
                query_update_points = '''
                    UPDATE mahasiswa
                    SET points_earned = points_earned + %s
                    WHERE id = %s
                '''
                cursor.execute(query_update_points, (event_point, student_id))
                commit()

                # Perbarui listbox dan treeview
                load_all_data_events_students()
                data_profile_student = list(select_items('*', "mahasiswa", f"WHERE id = {student_id}")[0][0])
                    header_student_profile = ['Id', 'Password', 'Nama', 'Prodi', 'Email', 'Nomor Kontak', 'Tahun Masuk',
'Total Point']
                create_profile_page(frame_student, data_profile_student, header_student_profile, 1, 0)

                messagebox.showinfo("Sukses", "Event berhasil dimasukkan ke log.")
```

```
            except Exception as e:
                messagebox.showerror("Error", f"Gagal menambahkan event ke log: {e}")
    else:
        messagebox.showerror("Error", "Event tidak ditemukan.")
```

Sebelum data dimasukkan, dilakukan pencarian dengan menggabungkan data dari log dengan mahasiswa dan *events* untuk memastikan bahwa kegiatan yang akan dimasukkan memang dapat dimasukkan (tersedia dan ada di dalam *database*) serta mahasiswa tersebut belum pernah mengikuti kegiatan tersebut.

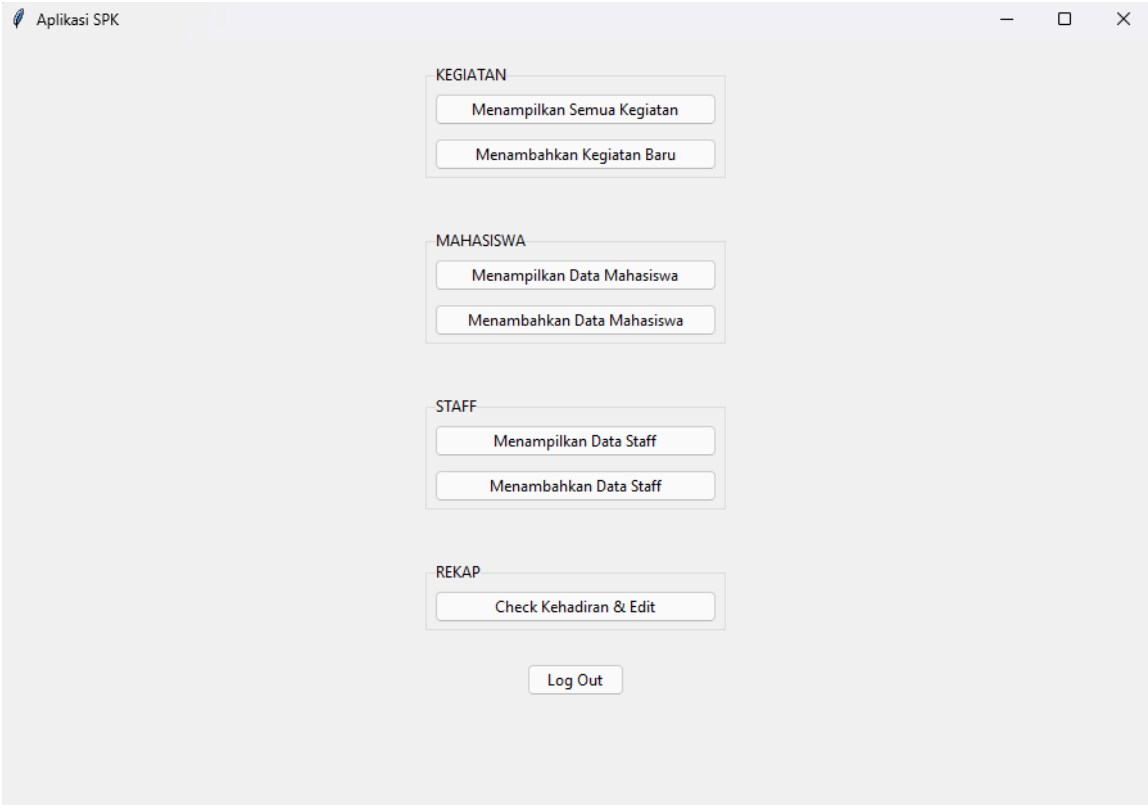
- Bagian untuk menampilkan data yang diambil dari database ataupun melakukan *refresh*:

```
def load_all_data_events_students():
    data = get_data_listbox_from_mysql_events_students("")  # Ambil semua data dari database
    # Filter untuk listbox (hanya event yang belum diikuti)
    listbox_data = [row for row in data if row[-1] == "Belum Diikuti"]
    # Refresh treeview
    display_data_events_students(data)  # Load data ke treeview

    # Refresh listbox
    frame_student_listbox.delete(0, tk.END)
    for row in listbox_data:
        frame_student_listbox.insert(tk.END, row[0])  # Masukkan nama event ke listbox

def display_data_events_students(data):
    for i in frame_student_tree.get_children():
        frame_student_tree.delete(i)  # Menghapus data lama
    for row in data:
        frame_student_tree.insert("", "end", values=row)

def display_listbox_data_events_students(options_events_students):
    frame_student_listbox.delete(0, tk.END)
    for option in options_events_students:
        frame_student_listbox.insert(tk.END, option[0])
```

Bagian ini penting untuk melakukan inisialisasi awal (dengan *load* data dari *database*) dan juga untuk ketika ada perubahan, fungsi ini akan dipanggil kembali untuk memastikan bahwa data yang ditampilkan adalah data yang paling baru. Karena load itu juga terdapat untuk penghapusan data yang ditampilkan dan menampilkan ulang, fungsi ini juga dapat digunakan sebagai fungsi *refresh*.

3.3. Halaman Admin

3.3.1. Inisialisasi penempatan semua menu



Gambar halaman *login admin*

```
frame_admin_button_width = 35
# menu berkaitan dengan event
frame_admin_label_event = ttk.LabelFrame(frame_admin, text='KEGIATAN')
frame_admin_label_event.grid(row=0, column=0, padx=20, pady=20)
frame_admin_button_show_event = ttk.Button(frame_admin_label_event, text='Menampilkan Semua Kegiatan',
                                           width=frame_admin_button_width, command=lambda: show_frame('show_events'))
frame_admin_button_show_event.grid(row=0, column=0, padx=5, pady=5)
frame_admin_button_new_event = ttk.Button(frame_admin_label_event, text='Menambahkan Kegiatan Baru',
                                          width=frame_admin_button_width, command=lambda: show_frame('new_events'))
frame_admin_button_new_event.grid(row=1, column=0, padx=5, pady=5)

# menu berkaitan dengan mahasiswa
frame_admin_label_mahasiswa = ttk.LabelFrame(frame_admin, text='MAHASISWA')
frame_admin_label_mahasiswa.grid(row=1, column=0, padx=20, pady=20)
frame_admin_button_show_mahasiswa = ttk.Button(frame_admin_label_mahasiswa, text='Menampilkan Data Mahasiswa',
                                               width=frame_admin_button_width, command=lambda: show_frame('show_students'))
frame_admin_button_show_mahasiswa.grid(row=0, column=0, padx=5, pady=5)
frame_admin_button_new_mahasiswa = ttk.Button(frame_admin_label_mahasiswa, text='Menambahkan Data Mahasiswa',
                                              width=frame_admin_button_width, command=lambda: show_frame('new_students'))
frame_admin_button_new_mahasiswa.grid(row=1, column=0, padx=5, pady=5)

# menu berkaitan dengan staff
frame_admin_label_staff = ttk.LabelFrame(frame_admin, text='STAFF')
frame_admin_label_staff.grid(row=2, column=0, padx=20, pady=20)
```

```
frame_admin_button_show_staff = ttk.Button(frame_admin_label_staff, text='Menampilkan Data Staff',
                                           width=frame_admin_button_width, command=lambda: show_frame('show_staffs'))
frame_admin_button_show_staff.grid(row=0, column=0, padx=5, pady=5)
frame_admin_button_new_staff = ttk.Button(frame_admin_label_staff, text='Menambahkan Data Staff',
                                          width=frame_admin_button_width, command=lambda: show_frame('new_staff'))
frame_admin_button_new_staff.grid(row=1, column=0, padx=5, pady=5)

# menu berkaitan dengan rekap
frame_admin_label_check_attendance = ttk.LabelFrame(frame_admin, text='REKAP')
frame_admin_label_check_attendance.grid(row=3, column=0, padx=20, pady=20)
frame_admin_button_check_attendance = ttk.Button(frame_admin_label_check_attendance, text='Check Kehadiran & Edit',
                                                 width=frame_admin_button_width, command=lambda: show_frame('check_attendance'))
frame_admin_button_check_attendance.grid(row=0, column=0, padx=5, pady=5)

frame_admin_logout = ttk.Button(frame_admin, text='Log Out', command=lambda: show_frame('login'))
frame_admin_logout.grid(row=4, column=0, columnspan=2, pady=5)
frame_admin.grid_columnconfigure(0, weight=1)
```

3.3.2. Pembagian bagian

Halaman Admin menampilkan 4 bagian menu yaitu bagian kegiatan, mahasiswa, *staff*, dan rekap. Bagian-bagian tersebut memiliki menu:

- Bagian kegiatan

Pada bagian kegiatan terdapat dua buah menu, yaitu menampilkan semua kegiatan dan menambahkan kegiatan baru. Menu menampilkan semua kegiatan dapat untuk melakukan pencarian informasi dari suatu kegiatan dan juga mengubah statusnya. Bagian mengubah statusnya ini yang akan menjadi poin U (*update*) kami. Menu menambahkan kegiatan baru akan menjadi poin C (*create*) kami.

- Bagian Mahasiswa dan *Staff*

Pada bagian mahasiswa dan *staff* sama-sama memiliki dua menu yaitu menampilkan data mahasiswa/*staff* dan menambahkan data mahasiswa/*staff* baru. Menu menampilkan mahasiswa/*staff* dapat untuk melakukan pencarian informasi dari seorang mahasiswa/*staff* dan juga ada opsi untuk menghapus data tersebut. Bagian menampilkan dan penghapusan ini akan menjadi point D (*delete*) kami. Menu menambahkan mahasiswa/*staff* baru akan menjadi poin C (*create*) kami.

- Bagian Rekap

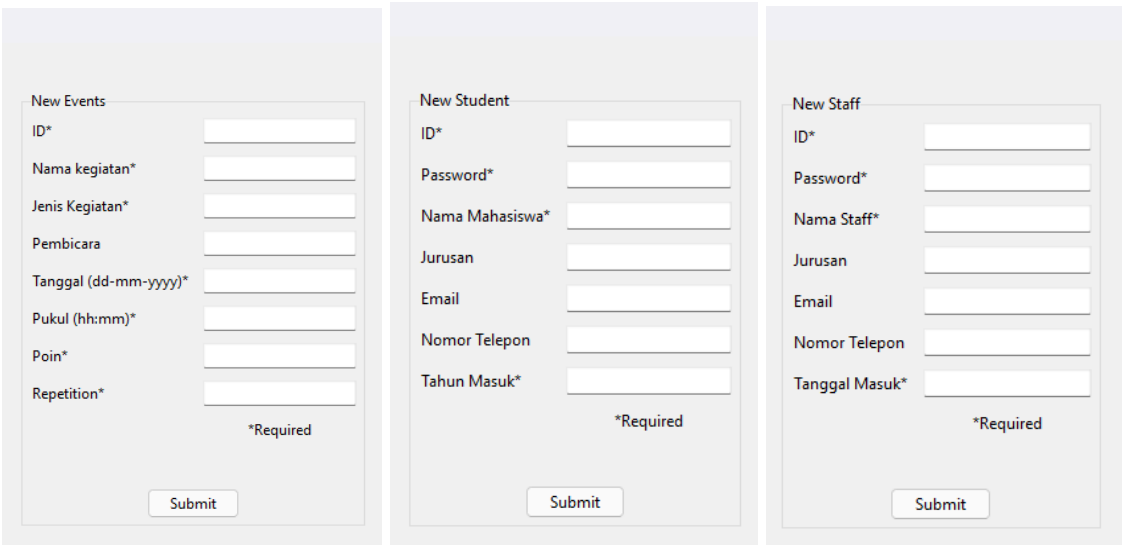
Pada bagian rekap terdapat satu menu yaitu untuk melakukan pengecekan kehadiran. Menu ini akan menampilkan dua buah diagram *pie* yang menunjukkan

banyaknya mahasiswa dan staff yang mengikuti suatu kegiatan dibandingkan dengan yang tidak mengikuti.

3.3.3. Pemenuhan Kriteria CRUD (Berserta penjelasan kode)

3.3.3.1. C (*create*)

- Halaman Admin: Menambahkan Data (*events*, mahasiswa, *staff*)



Gambar halaman menambahkan data *events,* mahasiswa, dan *staff.*

Untuk mendeklarasikan *entry* atau kotak buat mengetik, kami menggunakan *looping* dengan index untuk sistem peletakkan *grid*. Kami akan memberikan cara kerja dari menambahkan *event* untuk mewakili menambahkan mahasiswa ataupun *staff*. Berikut adalah cara kerjanya:

```
        for no, i in enumerate(("id","name","category","speaker","date","time","point","repetition")):
                                dictLabel       =       {"id":"ID*","name":"Nama       kegiatan*","category":"Jenis
Kegiatan*","speaker":"Pembicara","date":"Tanggal                                       (dd-mm-yyyy)*","time":"Pukul
(hh:mm)*","point":"Poin*","repetition":"Repetition*"}
            exec(f"frame_new_events_label_{i} = ttk.Label(frame_new_events_label, text='{dictLabel[i]}')")
            exec(f"frame_new_events_label_{i}.grid(row={no}, column=0, padx=5, pady=5, sticky='w')")
            exec(f"frame_new_events_entry_{i} = ttk.Entry(frame_new_events_label)")
            exec(f"frame_new_events_entry_{i}.grid(row={no}, column=1, padx=5, pady=5)")
```

Untuk memastikan bahwa input yang diberikan dalam format yang tepat, kami juga mengantisipasikan hal tersebut dengan memberikan validasi input. Berikut adalah validasinya:

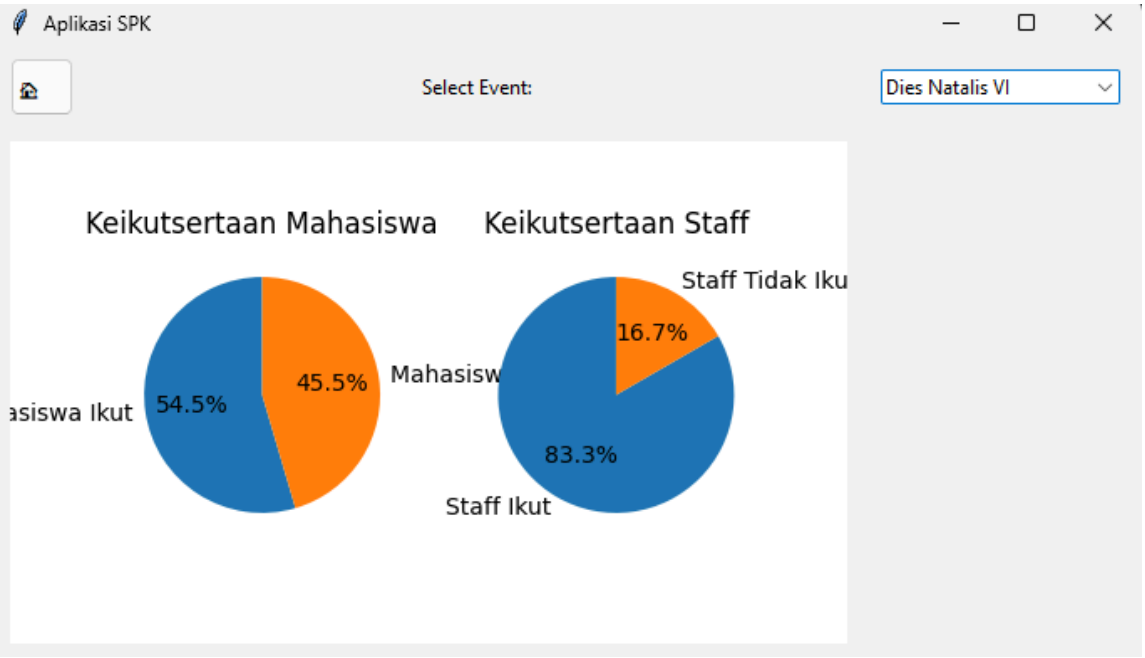
```
        def input_empty_validation(text, type):
```

```
            try:
                if len(text) == 0: raise ValueError
            except ValueError:
                    frame_new_events_label_result.config(text=f'System: Input kosong.\nSilakan memasukkan {type}!',
foreground="red", justify="center")
                    frame_new_students_label_result.config(text=f'System: Input kosong.\nSilakan memasukkan {type}!',
foreground="red", justify="center")
                    frame_new_staffs_label_result.config(text=f'System: Input kosong.\nSilakan memasukkan {type}!',
foreground="red", justify="center")
            else: return text

        def input_unique_validation(text, input_category, invalid_type):
            if input_category == "mahasiswa" or input_category == "staff":
                input_id = text.strip()
                if input_id in select_items("id", "mahasiswa")[0] or input_id in select_items("id", "staff")[0]:
                                frame_new_students_label_result.config(text=f'System: ID tidak valid karena sudah
dipakai.\nSilakan memasukkan ID yang unique!', foreground="red", justify="center")
                    frame_new_staffs_label_result.config(text=f'System: ID tidak valid karena sudah dipakai.\nSilakan
memasukkan ID yang unique!', foreground="red", justify="center")
                else: return input_id
            else:
                input_id = text.strip().lower()
                      if invalid_type == "ID" and input_id in list(map(lambda x: x.lower(), select_items("id",
"events")[0])):
                    frame_new_events_label_result.config(text=f'System: ID tidak valid karena sudah dipakai.\nSilakan
memasukkan ID yang unique!', foreground="red", justify="center")
                    elif invalid_type == "Name" and input_id in list(map(lambda x: x.lower(), select_items("name",
"events")[0])):
                                frame_new_events_label_result.config(text=f'System: Nama tidak valid karena sudah
dipakai.\nSilakan memasukkan nama lain yang unique!', foreground="red", justify="center")
                else: return text

        def input_date_validation(text):
            try:
                text = text.split('-')
                if len(text) > 3: raise IndexError
                for i in range(3):
                    text[i] = int(text[i])
            except ValueError:
                 frame_new_events_label_result.config(text="System: Input tidak valid.\nSilakan masukkan tanggal yang
benar.", foreground="red", justify="center")
                 frame_new_staffs_label_result.config(text="System: Input tidak valid.\nSilakan masukkan tanggal yang
benar.", foreground="red", justify="center")
            except IndexError:
                 frame_new_events_label_result.config(text="System: Input tidak valid.\nSilakan masukkan tanggal yang
benar.", foreground="red", justify="center")
                 frame_new_staffs_label_result.config(text="System: Input tidak valid.\nSilakan masukkan tanggal yang
benar.", foreground="red", justify="center")
            else:
                if not 1 <= text[0] <= 31 or not 1 <= text[1] <= 12 or not 1900 <= text[2] <= 2100:
                      frame_new_events_label_result.config(text="System: Input tidak valid.\nSilakan masukkan tanggal
yang benar.", foreground="red", justify="center")
                      frame_new_staffs_label_result.config(text="System: Input tidak valid.\nSilakan masukkan tanggal
yang benar.", foreground="red", justify="center")
                    return
                text[0] = '0' + str(text[0]) if len(str(text[0])) == 1 else str(text[0])
                text[1] = '0' + str(text[1]) if len(str(text[1])) == 1 else str(text[1])
                return '-'.join(map(str, text))

        def input_time_validation(text):
            try:
                text = text.split(':')
                if len(text) > 2: raise IndexError
                for i in range(2):
                    text[i] = int(text[i])
            except ValueError:
                    frame_new_events_label_result.config(text="System: Input tidak valid.\nSilakan masukkan jam yang
benar.", foreground="red", justify="center")
```

```
            except IndexError:
                    frame_new_events_label_result.config(text="System: Input tidak valid.\nSilakan masukkan jam yang
benar.", foreground="red", justify="center")
            else:
                if not 0 <= text[0] <= 23 or not 0 <= text[1] <= 59:
                     frame_new_events_label_result.config(text="System: Input tidak valid.\nSilakan masukkan jam yang
benar.", foreground="red", justify="center")
                    return
                text[0] = '0' + str(text[0]) if len(str(text[0])) == 1 else str(text[0])
                text[1] = '0' + str(text[1]) if len(str(text[1])) == 1 else str(text[1])
                return ':'.join(map(str, text))

        def input_float_validation(text):
            try:
                user_input = float(text)
                if user_input > 99.99 or user_input < 0: raise ValueError
                except ValueError: frame_new_events_label_result.config(text="System: Input tidak valid.\nSilakan
masukkan angka desimal yang benar.", foreground="red", justify="center")
            else: return round(user_input, 2)

        def input_int_validation(text):
            try:
                user_input = int(text)
                if user_input <= 0: raise ValueError
                except ValueError: frame_new_events_label_result.config(text="System: Input tidak valid.\nSilakan
masukkan angka bulat yang benar.", foreground="red", justify="center")
            else: return user_input

        def submit_new_events():
             if not (event_id := input_empty_validation(frame_new_events_entry_id.get(), 'ID')) or not (event_id :=
input_unique_validation(frame_new_events_entry_id.get(), 'events', "ID")): return
                if not (event_name := input_empty_validation(frame_new_events_entry_name.get(), 'nama')) or not
(event_name := input_unique_validation(frame_new_events_entry_name.get(), 'events', "Name")): return
                 if not (event_category := input_empty_validation(frame_new_events_entry_category.get(), 'kategori')):
return
            event_speaker = frame_new_events_entry_speaker.get()
            if not (event_date := input_date_validation(frame_new_events_entry_date.get())): return
            if not (event_time := input_time_validation(frame_new_events_entry_time.get())): return
            if not (event_point := input_float_validation(frame_new_events_entry_point.get())): return
            if not (event_repetition := input_int_validation(frame_new_events_entry_repetition.get())): return
                    frame_new_events_label_result.config(text="System: Kegiatan telah berhasil ditambahkan!",
foreground="green", justify="center")
            print(event_id.upper(), event_name, "available", event_category)
            insert_values("events", "id,name,status,category,speaker,date,time,point,max_repetition",
                              [(event_id.upper(), event_name, "available", event_category, event_speaker, event_date,
event_time, event_point, event_repetition)])
            commit()
            list_box_events()
            load_all_data_events()
            delete_entry_new_events()
```

3.3.3.2. R (*read*)

- Halaman Admin: Menampilkan rekapitulasi



Gambar halaman grafik rekapitulasi kehadiran

Pada halaman tersebut terdapat dua elemen yang penting, yaitu *combobox* untuk memilih kegiatan yang hendak ditampilkan dan diagram tersebut. Berikut adalah kode dari *combobox* dan fungsi yang berkaitan dengan *combobox*:

```
# Mendapatkan data events dari database
cursor.execute("SELECT id, name FROM events")
events = cursor.fetchall()

# Membuat combobox untuk memilih event
event_names = [event[1] for event in events]
event_ids = [event[0] for event in events]
selected_event_id = tk.StringVar()
selected_event_combobox = ttk.Combobox(frame_check_attendance, values=event_names, textvariable=selected_event_id)
selected_event_combobox.grid(row=0, column=2, padx=10)
selected_event_combobox.set("Select an event")

# Mengatur combobox callback untuk update pie chart
def on_event_select(event):
    selected_event_name = selected_event_combobox.get()
    if selected_event_name != "Select an event":
        update_pie_charts(selected_event_name)
```

Untuk menghasilkan diagramnya kami menggunakan tambahan modul yaitu `from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg` dan `import matplotlib.pyplot as plt`. Berikut adalah kode dari penampilan diagram yang dibagi menjadi beberapa bagian:

➔ Inisialisasi *canvas* untuk diagram *pie*

```
# Membuat canvas untuk menampilkan pie charts
canvas_frame = tk.Frame(frame_check_attendance)
canvas_frame.grid(row=1, column=0, columnspan=2, padx=10, pady=10)

canvas = FigureCanvasTkAgg(fig, canvas_frame)
canvas.get_tk_widget().grid(row=0, column=0)

# Set nilai default Combobox ke event pertama
if event_names:
    selected_event_combobox.set(event_names[0])
    update_pie_charts(event_names[0])

selected_event_combobox.bind("<<ComboboxSelected>>", on_event_select)
```

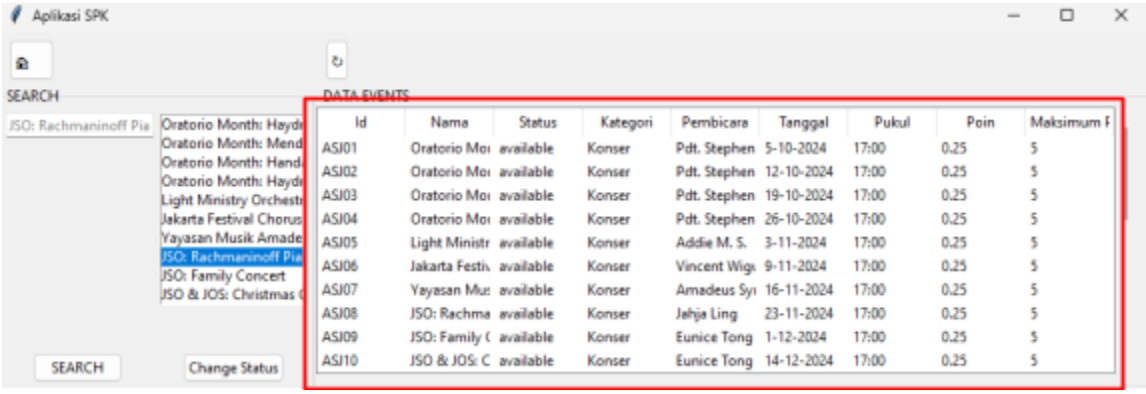
➔ Pengambilan data dari *database*

```
def update_pie_charts(event_name):
    # Mengambil data mahasiswa yang hadir
    cursor.execute(f"""
        SELECT m.id, m.name
        FROM mahasiswa m
        JOIN log_mahasiswa lm ON m.id = lm.id_mahasiswa
        JOIN events e ON lm.id_event = e.id
        WHERE e.name = %s
    """, (event_name,))
    mahasiswa_ikut = cursor.fetchall()

    # Mengambil total mahasiswa
    cursor.execute("""
        SELECT id, name FROM mahasiswa
    """)
    mahasiswa_total = cursor.fetchall()

    # Hitung mahasiswa yang tidak ikut
    mahasiswa_tidak_ikut = [m for m in mahasiswa_total if m not in mahasiswa_ikut]

    # Mengambil data staff yang hadir
    cursor.execute(f"""
        SELECT s.id, s.name
        FROM staff s
        JOIN log_staff ls ON s.id = ls.id_staff
        JOIN events e ON ls.id_event = e.id
        WHERE e.name = %s
    """, (event_name,))
    staff_ikut = cursor.fetchall()

    # Mengambil total staff
    cursor.execute("""
        SELECT id, name FROM staff
    """)
    staff_total = cursor.fetchall()

    # Hitung staff yang tidak ikut
    staff_tidak_ikut = [s for s in staff_total if s not in staff_ikut]
```

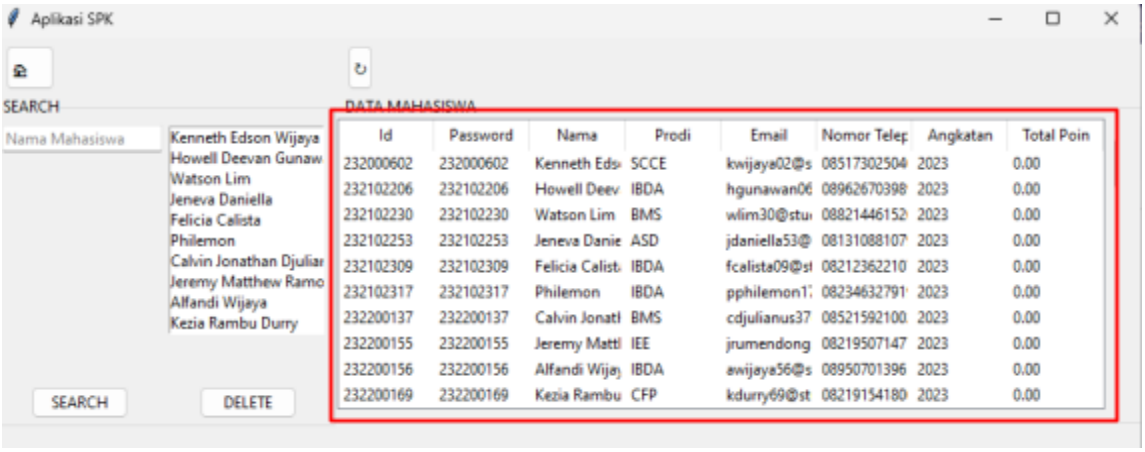
➔ Penghitungan mahasiswa/*staff* yang mengikuti dan menampilkan ke diagram

```
    # Update Pie Chart Mahasiswa
    mahasiswa_count = [len(mahasiswa_ikut), len(mahasiswa_tidak_ikut)]
    if sum(mahasiswa_count) == 0:  # Tidak ada data
        mahasiswa_count = [0, len(mahasiswa_total)]  # Semua dianggap tidak ikut
    ax1.clear()
        ax1.pie(mahasiswa_count,  labels=["Mahasiswa  Ikut",  "Mahasiswa  Tidak  Ikut"],  autopct='%1.1f%%',
startangle=90)
    ax1.set_title("Keikutsertaan Mahasiswa")

    # Update Pie Chart Staff
    staff_count = [len(staff_ikut), len(staff_tidak_ikut)]
    if sum(staff_count) == 0:  # Tidak ada data
        staff_count = [0, len(staff_total)]  # Semua dianggap tidak ikut
    ax2.clear()
    ax2.pie(staff_count, labels=["Staff Ikut", "Staff Tidak Ikut"], autopct='%1.1f%%', startangle=90)
    ax2.set_title("Keikutsertaan Staff")

    # Render canvas untuk pie chart
    canvas.draw()
```

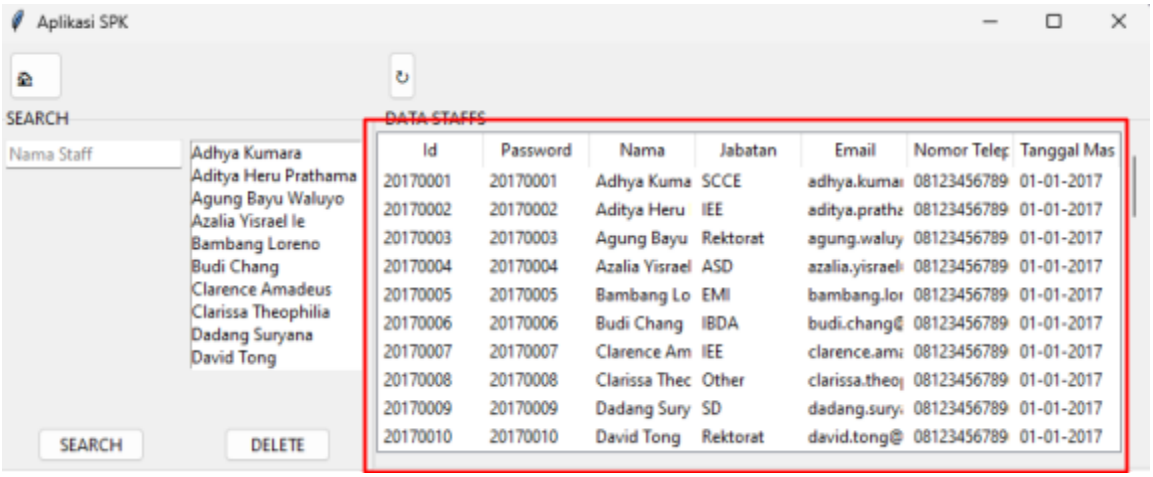
- Halaman Admin: Menampilkan Data (*events*, mahasiswa, *staff*)



Gambar menampilkan data *events* dan untuk mengganti status suatu kegiatan.



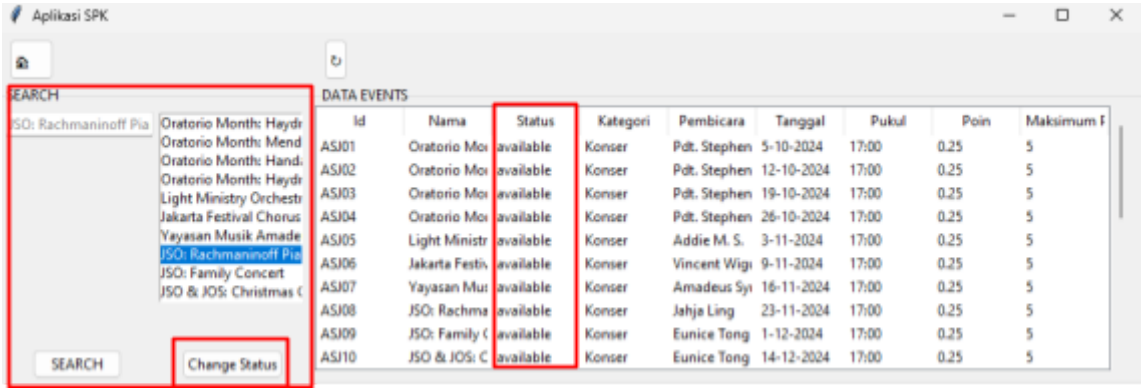
Gambar halaman menampilkan data mahasiswa dan untuk menghapus data mahasiswa yang dipilih.



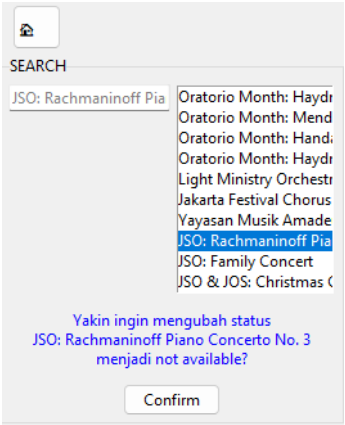
Gambar halaman menampilkan data *staff* dan untuk menghapus data *staff* yang dipilih.

3.3.3.3. U (*update*)

- Halaman Admin: Menampilkan Data (*events*)



Gambar menampilkan data *events* dan untuk mengganti status suatu kegiatan.



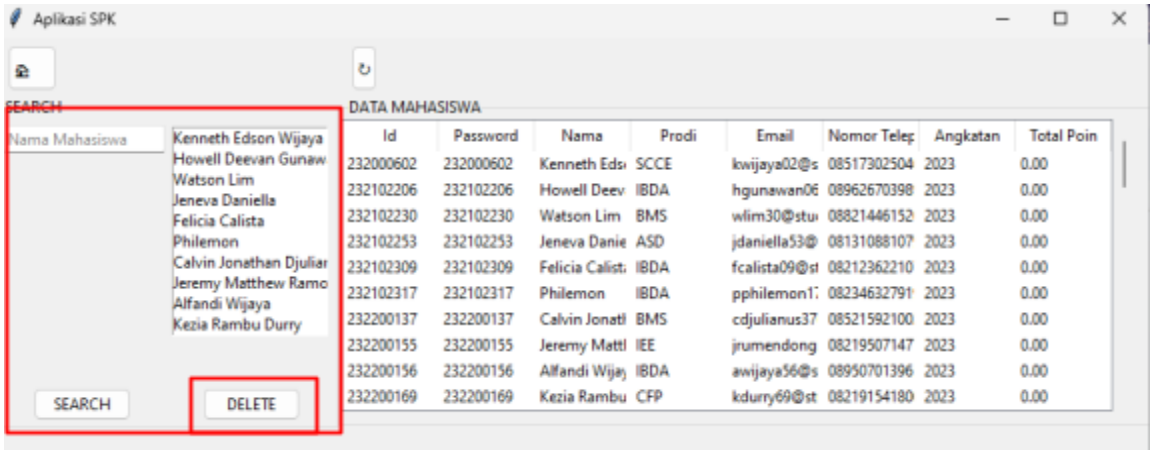
Gambar detail dari menampilkan data *events* dan untuk mengganti status suatu kegiatan.

Basis dari halaman ini sama dengan yang telah dijelaskan di awal untuk menampilkan tabel serta filternya, namun disini ada opsi untuk merubah status. Berikut adalah kode untuk perubahan status tersebut:

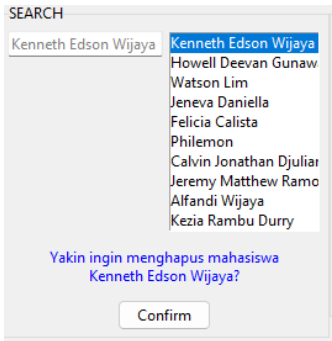
```
        def change_status_events():
            global frame_show_events_confirm_button
            name_events = entry_var_events.get()
            if name_events not in select_items("name", "events")[0]:
                        frame_show_events_label_result.config(text='Event tidak ditemukan', foreground="red",
justify="center")
                return

            def change_status_events_confirmed():
                    frame_show_events_label_result.config(text='Status berhasil diubah!', foreground="blue",
justify="center")
                frame_show_events_confirm_button.grid_forget()
                frame_show_events_submit_button.grid(row=2, column=0, padx=5, pady=5)
                frame_show_events_change_button.grid(row=2, column=1, padx=5, pady=5)
                    update_table('events', f'status = "{"not available" if current_status == "available" else
"available"}"', f"name = '{name_events}'")
                commit()
                load_all_data_events()
```

3.3.3.4. D (*delete*)

- Halaman Admin: Menampilkan Data (mahasiswa dan *staff*)



Gambar halaman menampilkan data mahasiswa dan untuk menghapus data mahasiswa yang dipilih.

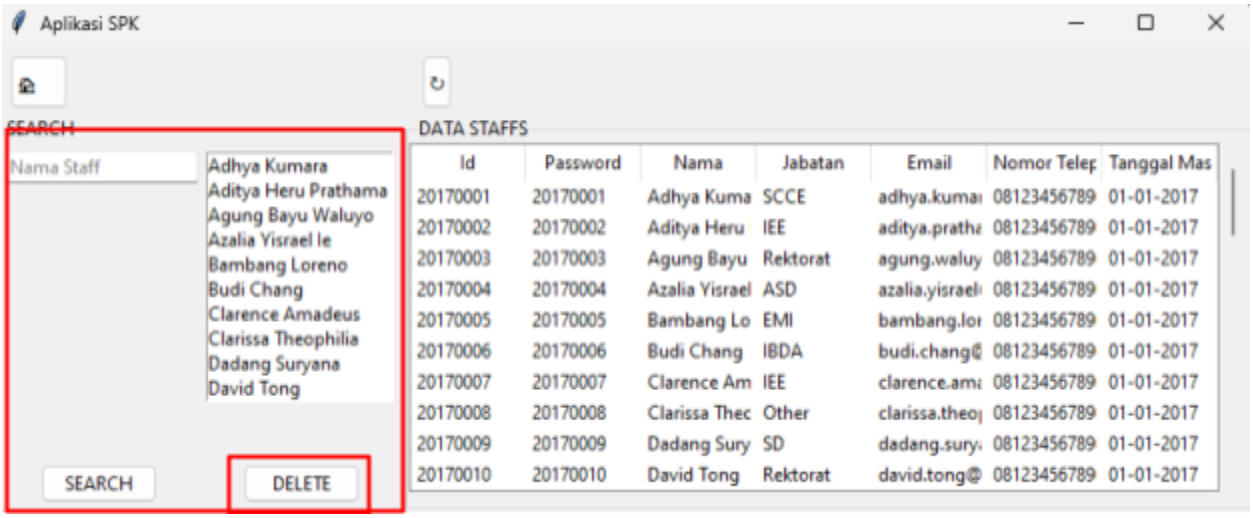


Gambar halaman

Basis dari halaman ini sama dengan yang sebelumnya untuk menampilkan tabel serta filternya, namun disini ada opsi untuk menghapus mahasiswa. Berikut adalah kode untuk penghapusan mahasiswa tersebut:

```
def delete_students():
    global frame_show_students_confirm_button
    name_students = entry_var_students.get()
    if name_students not in select_items("name", "mahasiswa")[0]:
        frame_show_students_label_result.config(text='Mahasiswa tidak ditemukan', foreground="red", justify="center")
        return

    def change_status_students_confirmed():
        frame_show_students_label_result.config(text='Mahasiswa berhasil dihapus!', foreground="blue", justify="center")
        frame_show_students_confirm_button.grid_forget()
        frame_show_students_search_button.grid(row=2, column=0, padx=5, pady=5)
        frame_show_students_delete_button.grid(row=2, column=1, padx=5, pady=5)
        print("Deleted Student")
        id_students = select_items('id', 'mahasiswa', f"where name = '{name_students}'")[0][0]
        delete_items("log_mahasiswa", f"id_mahasiswa = '{id_students}'")
        delete_items("mahasiswa", f"name = '{name_students}'")
        commit()
        list_box_students()
        load_all_data_students()
```

Kami juga memastikan bahwa penghapusan mahasiswa tersebut juga menghapus data yang disimpan di dalam log. Hal ini ditujukan agar hasil rekap diagram *pie* nya bisa sesuai jumlah mahasiswa yang ada.



Gambar halaman menampilkan data *staff* dan untuk menghapus data *staff* yang dipilih.

Sekian laporan pembuatan Sistem Poin Kemahasiswaan (SPK) dengan menggunakan basis bahasa pemrograman *python* dan *database mysql.* Proyek ini telah membawa kami kepada wawasan yang baru untuk menerapkan *python* dan *mysql* dalam memecahkan berbagai permasalahan di dunia nyata. Kami sadar bahwa proyek ini masih dapat dikembangkan di berbagai hal, termasuk fitur-fiturnya sehingga kami menantikan untuk dapat mengembangkan proyek ini lebih lanjut di dalam mata perkuliahan berikutnya.

Terimakasih sebesar-besarnya kepada Bapak Ghandy yang telah mengajarkan sistem *database mysql* dengan penerapannya menggunakan *python* dan membimbing kami di dalam proyek ini.